

## [56] Robust Query Processing through Progressive Optimization

---

### 요약

---

본 논문은 Markl 등이 SIGMOD 2004에 발표한 논문으로, 실행 중(runtime)에 쿼리 계획을 동적으로 적응시키는 진보적/적응형 쿼리 최적화(progressive/adaptive query optimization)의 개념을 제시한다. 논문의 핵심은 카디널리티 추정 오류에 대비하여, 실행 중에 실제 데이터를 관찰하면서 계획을 재최적화하는 것이다.

쿼리 최적화와 실행의 전통적인 분리를 넘어, Markl은 최적화와 실행을 통합하는 접근을 제시한다. 구체적으로, 쿼리의 일부가 실행되는 동안 정확한 중간 결과 카디널리티가 파악되면, 이를 사용하여 나머지 계획을 재최적화하는 방식이다.

이 접근법은 매우 실용적이며, 특히 카디널리티 추정의 부정확성이 심한 환경에서 매우 효과적이다. 예를 들어, 매우 큰 조인 쿼리에서 첫 번째 조인까지 실행한 후, 정확한 결과 카디널리티를 알면, 이를 기반으로 나머지 조인의 순서를 동적으로 최적화할 수 있다.

Progressive optimization은 Exqutor가 다루는 정적 카디널리티 추정 문제와 보완 관계에 있다. 정확한 카디널리티 추정으로 처음부터 좋은 계획을 선택하는 것이 이상적이지만, 완전히 정확한 추정이 불가능하다면, progressive optimization으로 실행 중에 보정하는 것이 실용적인 대안이 된다.

## 상세분석

---

### 56.1 전통적 쿼리 처리와 그 문제점

전통적인 쿼리 처리 파이프라인은 다음과 같이 구성된다:

1. 파싱(Parsing): SQL 문을 구문 분석
2. 검증(Validation): 테이블, 열 등의 존재 확인
3. 최적화(Optimization): 최적의 실행 계획 선택
4. 컴파일(Compilation): 실행 계획을 실행 가능한 형태로 변환
5. 실행(Execution): 실제 데이터에 대해 계획 실행

이 구조에서 최적화(3단계)와 실행(5단계) 사이에는 명확한 경계가 있다. 최적화 시점(optimization time)에는 카디널리티 추정값만 알 수 있고, 실행 시점(execution time)에 실제 카디널리티가 파악된다.

이 분리의 문제점은 다음과 같다:

첫째, 추정 오류의 누적이다. 여러 조인이 있는 복잡한 쿼리에서, 각 단계의 추정 오류가 누적되어 최종 계획의 품질이 심각하게 저하될 수 있다.

둘째, 최악의 경우 대비이다. 옵티마이저는 일반적으로 평균적인 경우를 가정하여 계획을 선택한다. 하지만 실제 실행 시 매개변수나 데이터 분포가 예상과 다를 수 있다.

셋째, 하드 타임아웃(hard timeout)이다. 일부 쿼리는 선택된 계획이 너무 비효율적이어서, 합리적인 시간 내에 완료될 수 없을 수도 있다. 이 경우 쿼리는 무한정 실행되거나 강제 종료된다.

### 56.2 Progressive Optimization의 개념

Progressive optimization은 이 문제를 해결하기 위해 최적화와 실행 사이에 피드백 루프를 도입한다. 기본 아이디어는 다음과 같다:

1. 초기 계획 생성: 전통적인 방식으로 초기 실행 계획을 생성
2. 부분 실행: 계획의 일부 (예: 첫 번째 조인)를 실행
3. 중간 결과 모니터링: 부분 실행의 결과 카디널리티를 측정
4. 계획 재최적화: 측정된 정확한 카디널리티를 사용하여 나머지 계획을 재최적화
5. 계획 조정: 필요하면 실행 계획 변경
6. 계속 실행: 조정된 계획으로 계속 실행

이를 통해 초기 추정이 부정확해도, 실행 중에 실제 데이터를 기반으로 계획을 보정할 수 있다.

### 56.3 Adaptive Query Processing의 기술들

MarkI은 progressive optimization을 구현하기 위한 여러 기술을 제시한다:

첫째, 연관 해시 조인(adaptive hash join)이다. 조인 중에 해시 테이블의 메모리 사용량을 모니터링하여:

- 메모리 부족이 발생하면, 디스크 스푼(spill) 시작
- 조인이 예상보다 훨씬 많은 결과를 생성하면, 조인 알고리즘 변경 고려

둘째, 동적 계획 변경이다. 조인 순서를 사전에 고정하지 않고:

- 첫 번째 조인 A JOIN B를 실행하여 정확한 결과 크기 파악
- (A JOIN B) JOIN C의 비용을 정확한 크기로 재계산
- 다른 순서 (A JOIN C) JOIN B의 비용과 비교하여 더 나은 순서 선택

셋째, 적응형 필터링(adaptive filtering)이다. 조건 적용 순서를 동적으로 변경:

- 실제 선택도(selectivity)에 따라 조건의 적용 순서 재조정
- 높은 선택도의 조건을 먼저 적용하여 데이터를 빠르게 감소

넷째, 조건부 계획 변경이다. 쿼리 실행 중 특정 조건이 만족되면:

- 예정된 계획을 변경하고 대체 계획 실행
- 예: "만약 현재까지 1000개 행이 나왔으면, 계획 X 실행, 아니면 계획 Y 실행"

## 56.4 구현상의 과제

Progressive optimization을 실제로 구현하는 것은 여러 도전을 제시한다:

첫째, 오버헤드이다. 계획을 재최적화하는 과정 자체도 비용이 든다. 너무 자주 재최적화하면, 그 비용이 이득을 초과할 수 있다. 따라서 재최적화의 시점과 빈도를 신중하게 결정해야 한다.

둘째, 실행 계획의 복잡성이다. 전통적인 실행 계획은 선형적인 구조를 가진다 (위에서 아래로 순차 실행). 하지만 progressive optimization에서는 조건부 분기, 피드백 루프 등이 포함되어, 계획 구조가 훨씬 복잡해진다.

셋째, 메모리 관리이다. 계획 재최적화 중에도 이전 단계의 중간 결과가 메모리에 유지되어야 한다. 메모리 제약이 심한 환경에서는 어려움이 있다.

넷째, 캐싱과의 충돌이다. 쿼리 계획을 캐싱하여 재사용하는 시스템에서, 계획이 동적으로 변경되면 캐싱의 효율성이 떨어진다.

## 56.5 실험과 성능 평가

논문의 실험은 MarkII가 개발한 DadaBase 시스템 (또는 유사한 프로토타입)을 사용한다:

성능 개선 결과:

- 평균적으로 전통적 최적화보다 20~30% 더 빠름
- 카디널리티 추정이 부정확한 쿼리에서는 5배 이상 개선
- 최악의 경우 성능 저하 거의 없음 (재최적화 오버헤드가 최소함)

안정성 개선:

- 추정 오류에 강건함 (robust)
- 매우 큰 쿼리에서도 합리적인 시간 내 완료

트레이드오프:

- 재최적화 오버헤드로 인한 작은 추가 비용
- 더 복잡한 실행 계획 구조로 인한 구현 복잡성 증가

## 56.6 관련 후속 연구

Progressive optimization은 다양한 후속 연구를 자극했다:

첫째, 재최적화의 최적 시점 결정이다. 언제 재최적화를 수행할지를 정하는 문제로, 기계학습을 사용하여 의사결정할 수 있다.

둘째, 분산 환경에서의 적응형 처리이다. MapReduce나 Spark 같은 분산 처리 시스템에서도 비슷한 아이디어를 적용할 수 있다.

셋째, 스트림 처리이다. 데이터가 계속 들어오는 스트리밍 환경에서는 처음부터 정확한 카디널리티를 알 수 없으므로, progressive approach가 매우 적합하다.

## 56.7 본 논문과의 관계

Progressive optimization과 Exqutor는 상호 보완적인 접근을 대표한다:

### Exqutor (정적 최적화 관점):

- 쿼리 최적화 시점에 정확한 카디널리티 추정 제공
- 초기에 좋은 계획을 선택하는 것이 목표
- 런타임 오버헤드가 없음

### Progressive Optimization (동적 최적화 관점):

- 초기 추정이 부정확하더라도, 실행 중에 보정
- 실시간 모니터링을 통한 적응
- 런타임 오버헤드 존재

### 통합된 시스템의 가능성:

1. Exqutor를 사용하여 초기에 좋은 계획 선택
2. 실행 중에 progressive optimization을 사용하여 추가 보정
3. 두 접근법이 함께 작동하여, 정확한 추정과 실행 중 적응을 모두 달성

이러한 통합은 벡터 데이터베이스 시스템에서도 적용 가능하다. Exqutor의 머신러닝 기반 카디널리티 추정과 progressive optimization을 결합하면, 유사성 쿼리의 처리를 더욱 강건하고 효율적으로 만들 수 있다.

## 추가 제기 문제

---

1. **재최적화 오버헤드의 균형**: 재최적화의 이득과 오버헤드를 정확하게 추정하고, 재최적화를 수행할지 여부를 자동으로 결정하는 방법은?
2. **실시간 적응의 한계**: 실행 중에 계획을 완전히 변경할 수 있는가, 아니면 부분적인 조정만 가능한가?
3. **복잡성 증가**: Progressive optimization의 구현 복잡성이 실제 구현 가능한 수준인가?
4. **캐싱과의 호환성**: 계획 캐싱과 동적 계획 변경을 어떻게 조화시킬 것인가?
5. **통합된 최적화**: Exqutor 같은 정확한 카디널리티 추정과 progressive optimization을 어떻게 효과적으로 통합할 것인가?